

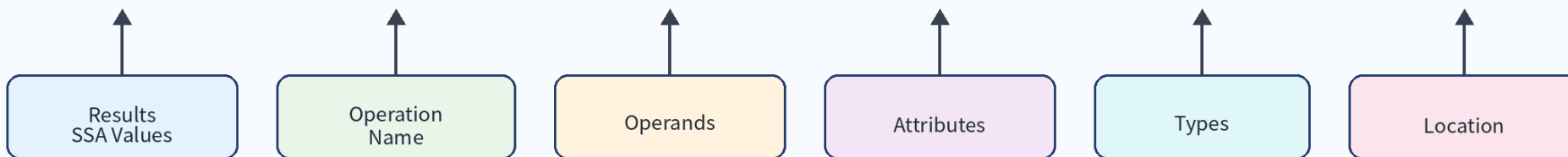
Lecture 02

MLIR IR Basic Structure

Operation / Value / Type / Attribute 를 NPU Compiler 관점에서 읽기

MLIR Operation Anatomy - Lecture 02

```
%out, %tok = "npu.matmul"(%A, %B, %acc) {  
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32  
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)  
  -> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```



NPU compiler reading order: op name -> operands/results -> types -> attributes -> location -> verifier constraints

해석 포인트: value는 SSA handle, type은 shape/element/layout contract, attribute는 compile-time metadata입니다.

2 강의 위치

1 강의 큰 그림에서 실제 IR 읽기로 내려옵니다.

1 강
전체 pipeline

2 강
IR 구성요소

3 강
Region/Block

7 강 +
Linalg/Tensor

12 강 +
Buffer/MemRef

18 강 +
Custom NPU
Dialect

- 오늘의 목표 : MLIR 한 줄을 compiler data structure 로 분해
- 핵심 산출물 : Operation/Value/Type/Attribute annotation 표
- NPU 연결 : fusion, tiling, bufferization, memory planning 의 입력 정보 식별

Learning Objectives

강의 종료 후 할 수 있어야 하는 것

- MLIR operation 의 results / operands / attributes / type signature / location 을 구분한다 .
- SSA value 의 definition 과 use 를 추적한다 .
- tensor, memref, vector, index, scalar type 이 NPU pipeline 에서 갖는 의미를 설명한다 .
- attribute 와 runtime operand 를 구분한다 .
- MatMul 예제 IR 을 NPU lowering 관점에서 audit 한다 .

MLIR IR 의 최소 구성 단위

Operation 중심으로 value/type/attribute 가 결합됩니다 .

Operation
변환의 기본 단위
ex) linalg.matmul

Value
SSA result 또는 block arg
ex) %out

Type
shape/element/layout
contract
ex) tensor<...>

Attribute
compile-time metadata
ex) tile=[...]

Location
source provenance
ex) loc(...)

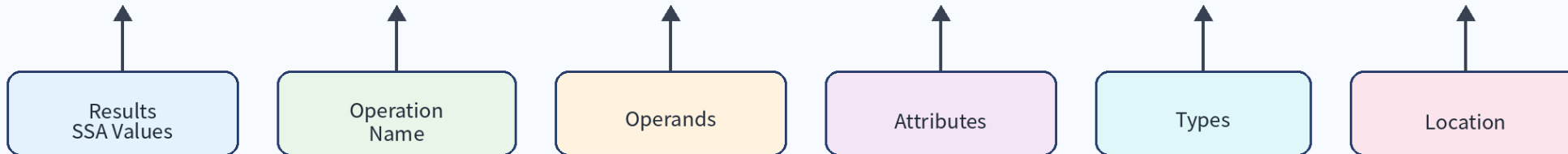
- Operation 이 IR graph 의 node 라면 , Value 는 edge/handle 입니다 .
- Type 은 edge 의 계약 , Attribute 는 node 에 붙은 static metadata 입니다 .
- NPU compiler 는 이 정보를 읽어 legality, schedule, memory plan 을 결정합니다 .

Operation Anatomy

한 줄의 IR 에서 무엇을 분리해 읽을 것인가

MLIR Operation Anatomy - Lecture 02

```
%out, %tok = "npu.matmul"(%A, %B, %acc) {  
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32  
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)  
  -> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```



NPU compiler reading order: op name -> operands/results -> types -> attributes -> location -> verifier constraints

해석 포인트: value는 SSA handle, type은 shape/element/layout contract, attribute는 compile-time metadata입니다.

Generic Operation Syntax

custom form 이 헛갈릴 때 generic form 으로 구조를 확인합니다 .

```
// Custom assembly form: concise and dialect-specific.
```

```
%0 = arith.addf %a, %b : f32
```

```
// Generic assembly form: explicit operation name, operands, attributes, and function type.
```

```
%0 = "arith.addf"(%a, %b) : (f32, f32) -> f32
```

- Generic form = "dialect.operation"(operands) {attributes} : input-types -> result-types
- custom NPU dialect 초기 bring-up 에서는 generic form 이 debug 에 유리합니다 .
- 이후 자주 쓰는 op 만 custom parser/printer 로 가독성을 높이면 됩니다 .

Operation Field Checklist

NPU pass writer 가 보는 필드

name	dialect + op semantic	legalization target
operands/results	SSA values	def-use / dependency
attributes	static metadata	schedule/layout/quant
regions	nested IR	loop/body/kernel
location	source mapping	debug/profiling

SSA Value 읽기

value 는 이름이 아니라 use-def edge 입니다 .

```
%zero = arith.constant 0.0 : f32          // op result value
%init = tensor.empty() : tensor<4x16xf32> // op result value
%filled = linalg.fill ins(%zero : f32)    // %zero is used here
      outs(%init : tensor<4x16xf32>) -> tensor<4x16xf32>
%out = linalg.matmul ins(%lhs, %rhs : tensor<4x8xf32>, tensor<8x16xf32>)
      outs(%filled : tensor<4x16xf32>) -> tensor<4x16xf32>
```

• %zero: arith.constant 결과 , linalg.fill 에서 사용

- %init: tensor.empty 결과 , allocation 후보
- %filled: fill 결과 , matmul destination 으로 사용
- %out: matmul 결과 , function return 으로 전달

Type 은 계약이다

shape, element, memory, vector/token 의미를 다음 pass 에 전달

tensor<1x128x768xf16>
ranked tensor
model/tensor IR

memref<128x768xf16, #layout, 1>
buffer + layout
memory space

vector<16x16xf16>
register/tile vector
tensorization

!npu.token
custom dependency
DMA/compute ordering

- Type mismatch 는 lowering 오류의 가장 빠른 signal 입니다 .
- NPU 에서는 accumulator width, memory space, vector tile shape 를 type 또는 descriptor 로 명확히 표현해야 합니다 .
- Dynamic shape 는 type 만으로 충분하지 않을 수 있어 runtime descriptor/shape value 가 필요합니다 .

Attribute ≡ compile-time metadata

static decision 과 runtime value 를 구분합니다 .

```
#map0 = affine_map<(d0, d1, d2) -> (d0, d2)>
#map1 = affine_map<(d0, d1, d2) -> (d2, d1)>
#map2 = affine_map<(d0, d1, d2) -> (d0, d1)>

"linalg.generic"(%A, %B, %C) {
  indexing_maps = [#map0, #map1, #map2],
  iterator_types = ["parallel", "parallel", "reduction"]
} : (tensor<4x8xf32>, tensor<8x16xf32>, tensor<4x16xf32>) -> tensor<4x16xf32>
```

- indexing_maps / iterator_types: structured op semantics
- tile / layout / memory_space: NPU schedule 또는 placement decision
- quant scale: static 이면 attribute, runtime 이면 operand/descriptor
- pointer address / dynamic batch: attribute 가 아니라 runtime data

Location 을 버리지 마세요

debug/profiling/accuracy 추적의 연결고리

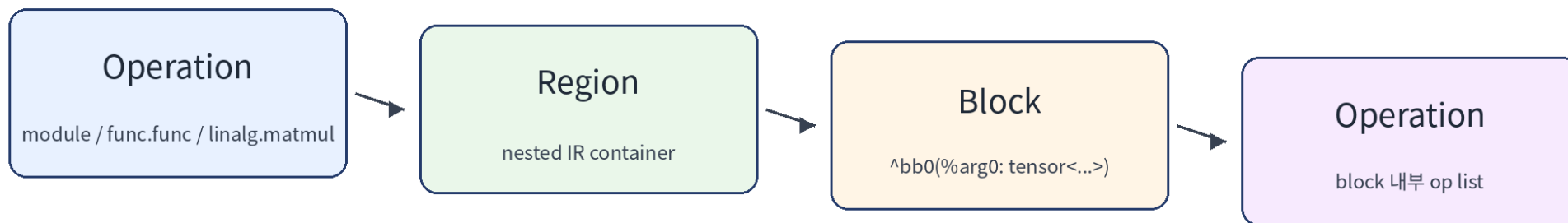
```
%0 = arith.addf %a, %b : f32 loc("model.py":42:7)
%1 = "npu.matmul"(%A, %B) : (...) -> (...) loc("encoder.layer.3.mlp.fc1")
```

- Lowering 후에도 어떤 모델 node 에서 온 op 인지 추적해야 합니다 .
- 성능 hotspot 을 source graph 로 되돌려야 schedule 개선이 가능합니다 .
- 정확도 문제는 quantized op 와 원래 op 를 매칭해야 디버깅할 수 있습니다 .

Operation-Region-Block 재귀 구조

3 강의 주제지만 , 2 강에서도 전체 그림은 필요합니다 .

MLIR IR Nesting Model



Lecture 02: Operation / Value / Type / Attribute를 읽습니다. Lecture 03: Region / Block을 더 깊게 다룹니다.

NPU lowering에서는 이 구조를 순회하며 fusion 후보, bufferization 경계, scheduling region을 찾습니다.

Example: linalg-on-tensors MatMul

operation/result/type 을 실제 snippet 에서 읽기

```
module {
  func.func @matmul_basic(
    %lhs: tensor<4x8xf32>,
    %rhs: tensor<8x16xf32>) -> tensor<4x16xf32> {
    %zero = arith.constant 0.000000e+00 : f32
    %init = tensor.empty() : tensor<4x16xf32>
    %filled = linalg.fill ins(%zero : f32)
      outs(%init : tensor<4x16xf32>) -> tensor<4x16xf32>
    %out = linalg.matmul
      ins(%lhs, %rhs : tensor<4x8xf32>, tensor<8x16xf32>)
      outs(%filled : tensor<4x16xf32>) -> tensor<4x16xf32>
    func.return %out : tensor<4x16xf32>
  }
}
```

MatMul 예제 읽기 순서

IR audit 을 pass writer 처럼 수행

- func.func: function body 를 가진 operation, %lhs/%rhs 는 block argument
- arith.constant: scalar f32 value 생성
- tensor.empty: destination tensor 생성 , bufferization 이후 allocation 후보
- linalg.fill: zero init. matmul init folding 또는 fill 제거 후보
- linalg.matmul: structured matmul semantic. tiling/vectorization/tensorization 대상
- func.return: result type 일치 검증

NPU Compiler IR Audit Checklist

Operation/Value/Type/Attribute 에서 pass 입력 정보 뽑기

Operation level	graph/tensor/buffer/kernel/runtime 중 어느 level 인가 ?
Def-use	consumer 수와 alias 가능성은 ?
Type legality	shape/element/memory space 가 target 과 맞는가 ?
Attribute legality	tile/layout/quant 가 capability 와 맞는가 ?
Dynamic shape	runtime descriptor 또는 guard 가 필요한가 ?

Attribute vs Operand Decision

static 이면 attribute, runtime 이면 operand/descriptor

tile size [16,16,64]

static weight quant scale

runtime batch size

activation pointer address

DMA dependency

SRAM memory space

Attribute

Attribute

Operand/shape value

Runtime descriptor

Value/token

Type/attribute

Custom NPU Dialect 설계 관점

2 강의 개념은 18 강 custom dialect 의 기초입니다 .

```
%desc = "npu.make_desc"(%buffer, %shape) {layout = "NHWC", space = "sram"} : (...) -> !npu.desc
%tok0 = "npu.dma"(%src, %dst, %desc) {kind = "h2s"} : (...) -> !npu.token
%out, %tok1 = "npu.matmul"(%A, %B, %acc, %tok0) {tile = [16,16,64]} : (...) -> (... , !npu.token)
```

- descriptor 는 runtime shape/pointer 정보를 담을 수 있습니다 .
- token value 는 DMA 와 compute 의 ordering 을 표현합니다 .
- tile 은 compile-time schedule 이면 attribute 가 자연스럽습니다 .

실습 A: Operation Anatomy Annotation

pseudo npu.matmul 을 구성요소별로 표시

```
%out, %token = "npu.matmul"(%A, %B, %acc) {  
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32  
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)  
  -> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```

- Results, operation name, operands, attributes, operand types, result types, location 을 표시
- tile 과 memory_space 가 attribute 일 때의 장단점 작성
- !npu.token 이 필요한 상황을 하나 이상 설명

실습 B: Def-Use Chain

value table 을 작성하고 bufferization 후보를 찾기

- 대상 value: %lhs, %rhs, %zero, %init, %filled, %out
- 각 value 에 대해 definition op, type, use op 를 표로 작성
- %init/%filled/%out 이 같은 buffer 로 alias 될 수 있는 조건 토론
- linalg.fill 제거 또는 matmul init folding 가능 조건 작성

실습 C: Type / Attribute Audit

NPU target constraint 를 IR 에서 찾기

- INT8 input, i32 accumulator, INT8 output requant pipeline 의 type 변화를 적기
- DRAM/SRAM/Accumulator memory space 를 type/attribute/descriptor 중 어디에 둘지 결정
- Dynamic batch 또는 dynamic sequence length 가 있는 경우 operand/descriptor 가 필요한 지점 찾기
- Verifier rule 3 개 작성 : shape, element type, tile divisibility 등

Common Mistakes

초기 MLIR/NPU compiler 개발에서 자주 보이는 문제

- runtime 값을 attribute 로 고정해 dynamic shape 대응이 불가능해짐
- layout/memory space 를 type 에도 attribute 에도 남기지 않아 DMA legality 판단 실패
- rewrite 후 old value 의 use 를 남겨 verifier 실패
- location 을 잃어 profiling hotspot 을 모델 node 로 되돌릴 수 없음
- custom syntax 만 읽고 generic type signature 를 놓침

Mini Quiz

팀별로 5 분 토론

- operation result 와 block argument 의 차이는 ?
- runtime pointer address 를 attribute 로 두면 왜 위험한가 ?
- tensor<1x128x768xf16> 에서 shape 와 element type 은 ?
- generic assembly 가 custom NPU dialect debug 에 유리한 이유는 ?
- npu.matmul verifier rule 을 하나 제안하라 .

과제

다음 강의 Region/Block 으로 연결

- 작은 MatMul/Add/ReLU 그래프를 StableHLO 또는 TOSA 스타일로 작성하고 op/value/type/attribute 표를 만든다 .
- 같은 연산을 Linalg-on-Tensors 로 표현할 때 type/attribute 변화 설명
- npu.matmul, npu.dma, npu.barrier pseudo op 를 generic assembly 스타일로 설계
- Region/Block 이 필요한 op 를 3 개 찾고 이유 설명

Takeaways

2 강에서 반드시 가져갈 것

- Operation 은 MLIR transformation 의 기본 단위입니다 .
- Value 는 SSA handle 이고 , Type 은 그 value 의 의미 계약입니다 .
- Attribute 는 compile-time metadata 이며 runtime 값과 분리해야 합니다 .
- Generic assembly 를 읽으면 op 구조가 명확해집니다 .
- NPU compiler 의 fusion/tiling/bufferization/memory planning 은 모두 이 구조에서 출발합니다 .

References

공식 문서 중심

- MLIR Language Reference
- Understanding the IR Structure
- Toy Tutorial Chapter 2: Emitting Basic MLIR
- Builtin Dialect
- Func Dialect
- Linalg Dialect